Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo

Sustainable Digital

MINI-PROJECT HACKATHON: “THE GREENEST WEBSITE” : BIGFOOT

Part 1: Scoping, Initial Design and Collaboration

Phase 1 — Product Scoping

1. Identify the Need and the Value Proposition

User Need: Individuals often lack a clear, quantifiable understanding of their personal carbon footprint and do not know which specific lifestyle changes will yield the most significant ecological impact.
Website Type: An educational and analytical web platform (Carbon Footprint Calculator).
Target Audience: The general public, specifically individuals looking to understand, track, and reduce their environmental impact.
Value Proposition: BIGFOOT provides a simple, guided 5-step calculator that not only estimates annual CO2 emissions but also delivers dynamic, customized mitigation advice based on the user's highest-emitting category.
Realistic Scope: The project is highly feasible for a 4-5 student team. By relying on a serverless architecture with Supabase as a BaaS, the team can bypass complex backend infrastructure setup and focus entirely on core logic, accurate API calculations (Rest Countries API), and an optimized, lightweight frontend DOM manipulation.

2. Define the MVP

Core Function: The 5-step footprint calculator (Transport, Housing, Food,. Consumption, Waste) utilizing predefined emission factors to output a total score in kg CO_{2}/year.
Essential Features (Usable MVP):

- Form collection for the 5 categories.
- Calculation logic (including the Haversine formula for flights).
- Results page with a DOM-based bar chart and dynamic feedback.
- User authentication (Supabase Auth) to save and view historical assessments.
- Admin dashboard for global overviews.

Postponed Features (V2): Social sharing of scores, gamification (badges for lowering emissions), and detailed sub-category tracking (e.g., specific grocery item logging).

3. Define Users and Their Roles

Guest (Unregistered): Can view the homepage and potentially test the calculator, but cannot save their results.
Registered User:

- *Create:* Can create an account and generate new footprint assessments.
- *Read:* Can view their personal dashboard and past assessment history.
- *Update/Delete:* Can delete specific historical records they own.

Administrator:

- *Read:* Can access the admin.html panel to view global platform metrics (total assessments, average footprint) and list all registered users.
- *Delete:* Has moderation tools to delete any user and their associated score history entirely.

4. Identify the Data to Be Managed

Main Business Entity (Bilans): This table stores the footprint data. Fields include id, user_id, date, transport_score, housing_score, food_score, consumption_score, waste_score, and total_score.
User Data (Profiles): Handled via Supabase. Fields include id (UUID), email, encrypted password (managed by Auth), and a boolean flag is_admin.
Data Relationships: A One-to-Many relationship exists between Users and Assessments (one user can have multiple saved bilans).
Sobriety Constraint: To remain lean, we do not store profile pictures, complex personal biographies, or excessive granular data (like the exact model of car). We only store the calculated numerical scores for each category to minimize database size and query payloads.

5. Write User Stories

User Management:

- As a user, I want to create a secure account so that I can save my footprint history and track it over time.
- As an administrator, I want to be able to delete users and their data so that I can moderate the platform and comply with data removal requests.

Main functionality:

- As a user, I want to input my lifestyle habits across 5 categories so that the application can calculate my estimated annual carbon footprint.
- As a user, I want to see dynamic, personalized advice based on my highest-emitting category *so that* I know exactly where to focus my reduction efforts.

Green IT / User experience :

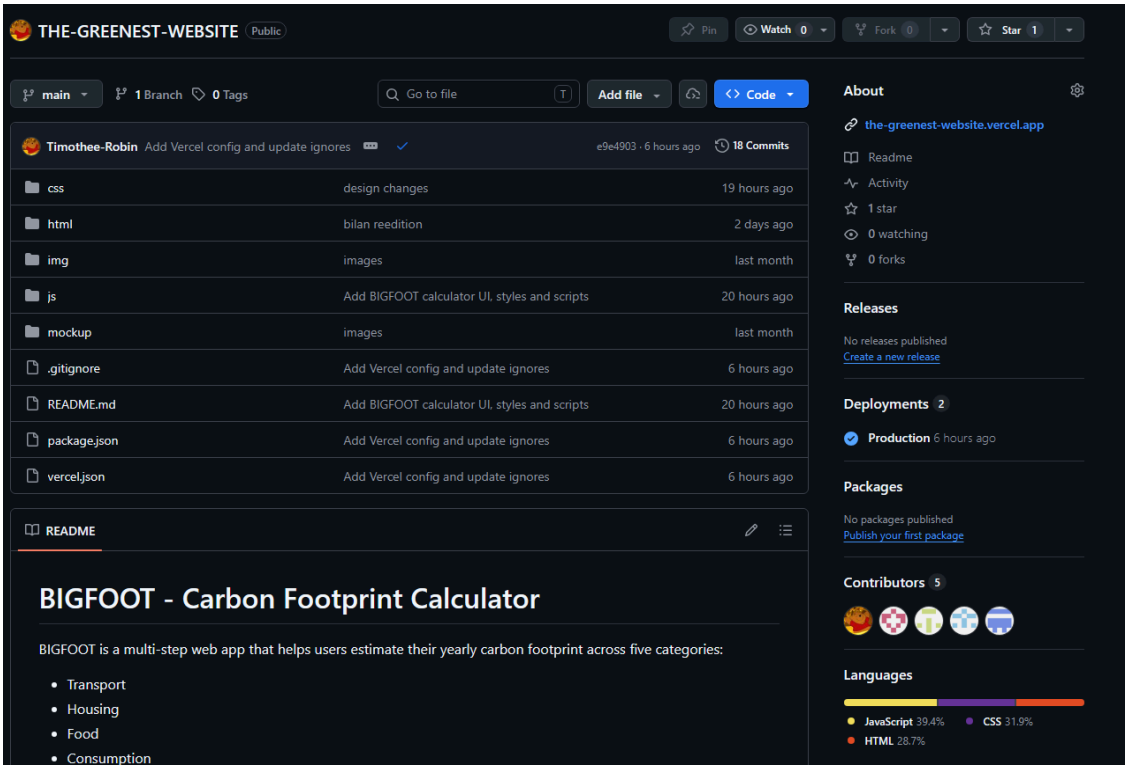
- As a user, I want the interface to load instantly without heavy media or unnecessary animations so that my browsing experience remains smooth and environmentally friendly.

6. Build a Mini Backlog

ID	Feature	Priority	Difficulty	Assignee
T01	Setup GitHub repo, branching rules, & Vercel deployment	High	Low	Timothée
T02	Initialize Supabase project & define database schemas	High	Medium	Youcef
T03	Develop pure HTML/CSS layout (index, bilan, result, admin)	High	Medium	Timothée
T04	Implement 5-step calculator logic (Vanilla JS) & RestCountries API	High	High	Ibrahima
T05	Integrate Supabase Auth (Sign up, Login, Session state)	High	Medium	Clayton
T06	Implement Assessment CRUD (Save score, view history, delete)	High	High	Clayton
T07	Build DOM-based results chart and dynamic feedback logic	Medium	Medium	Ibrahima
T08	Develop Admin dashboard (Global stats & User	Medium	High	Jeremy

	deletion)			
T09	Parcel build pipeline configuration & asset optimization	Low	Medium	Timothée

Phase 2 — Team Organization and Collaboration with GitHub



Phase 3 — Functional and Technical Design

1. Functional Architecture

Our system architecture relies on a lean, serverless model designed to minimize intermediary server processing and reduce latency.

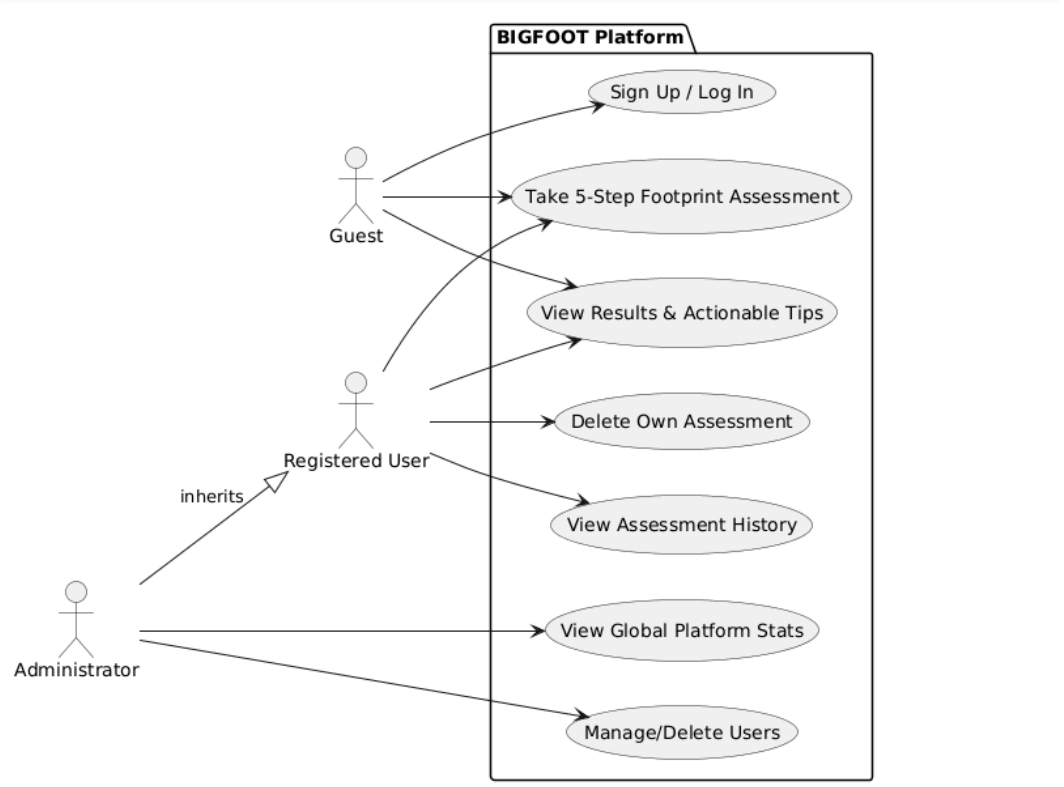
- **System Actors:**
  - **Guest:** Can calculate a footprint but cannot save it.
  - **Registered User :** Can calculate, save, view, and delete their own footprint history.
  - **Administrator:** Can access global platform metrics and moderate (delete) users and their data.
- **Main Features:**
  - 5-Step Carbon Footprint Calculator (Transport, Housing, Food, Consumption, Waste).
  - Dynamic Results Dashboard (DOM-based visualizations and customized reduction tips).
  - User Authentication & History Tracking.
  - Admin Moderation Panel.
- **Major Components:**
  - **Frontend Logic:** Native HTML5, CSS3, and Vanilla JavaScript. Forms are processed locally in the browser to reduce server round-trips.
  - **Backend & Database (BaaS):** Supabase handles both Auth and the PostgreSQL database. Queries are executed directly from the frontend via the Supabase JS Client.

- **External Integration:** Rest Countries API, used exclusively when flight calculations require geocoding for the Haversine formula.

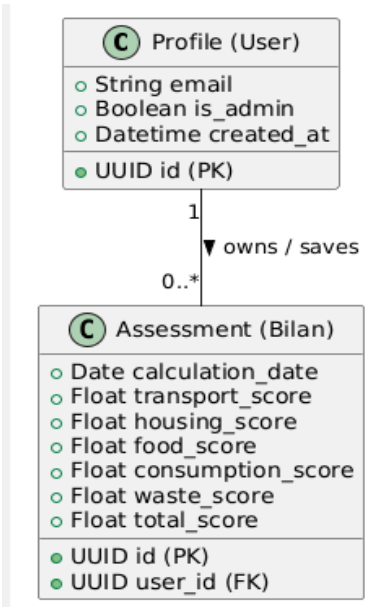
- **Essential Data:**
  - Users (Managed by Supabase Auth).
  - Profiles (Role management, e.g., is_admin).
  - Bilans (Historical footprint records linked to users).
- **Major Interactions:** The browser acts as the primary orchestrator. It manages state via localStorage during the 5-step form, directly queries the Rest Countries API for location data, computes the final payload via Vanilla JS, and pushes the data directly to Supabase via asynchronous REST calls.

2. Expected Diagrams

Use case Diagram:

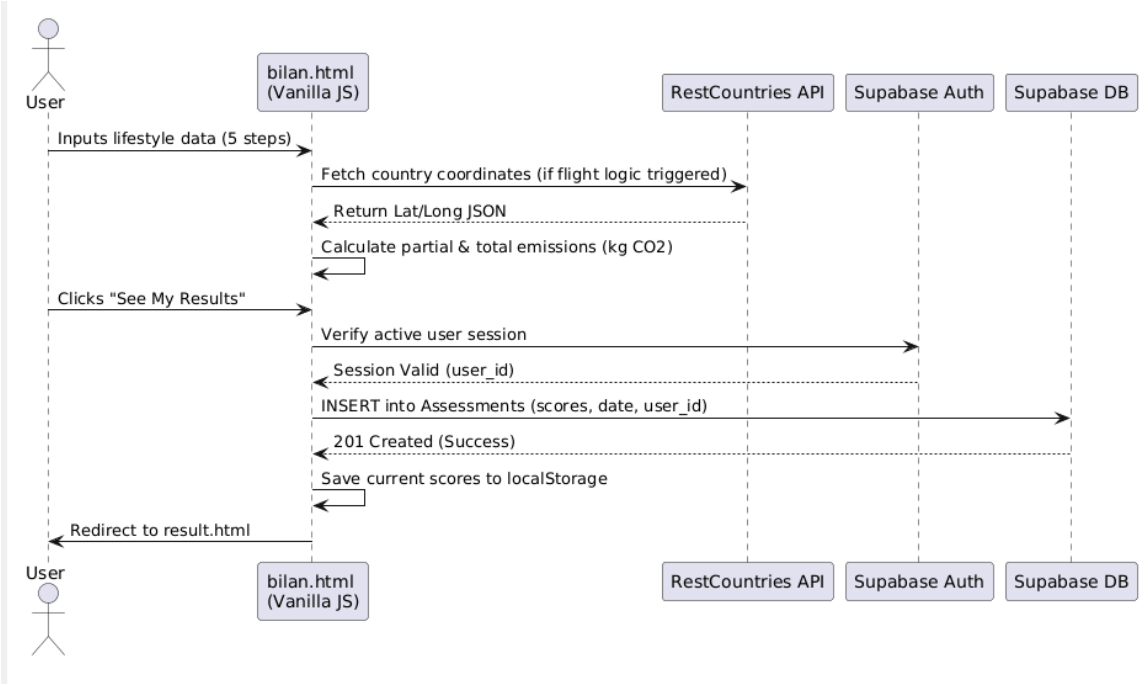


Class Diagram

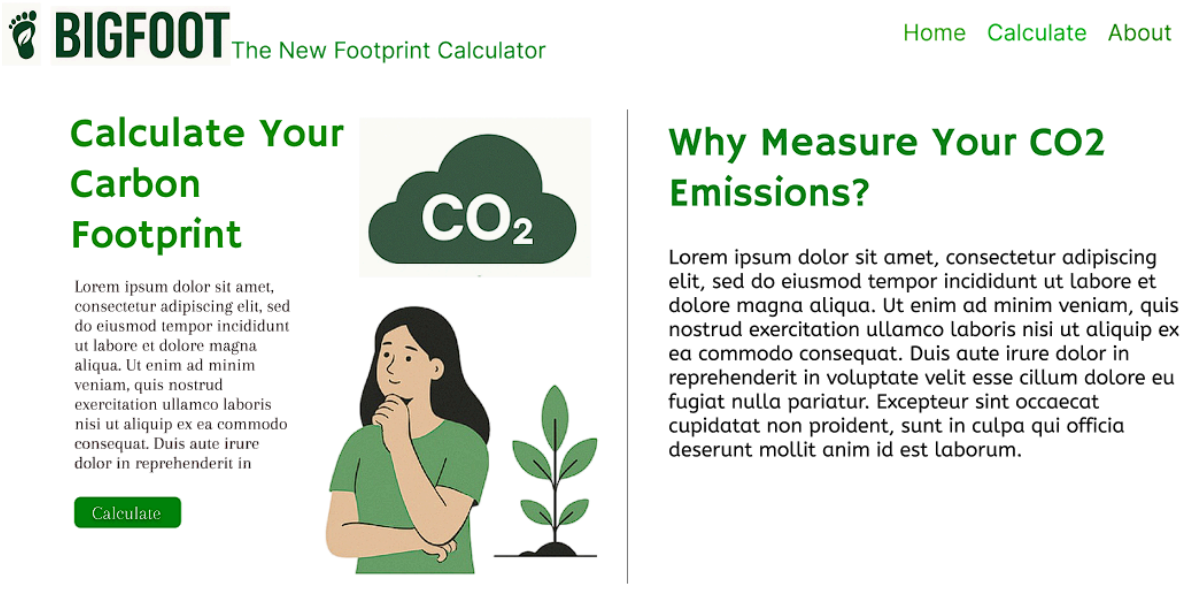


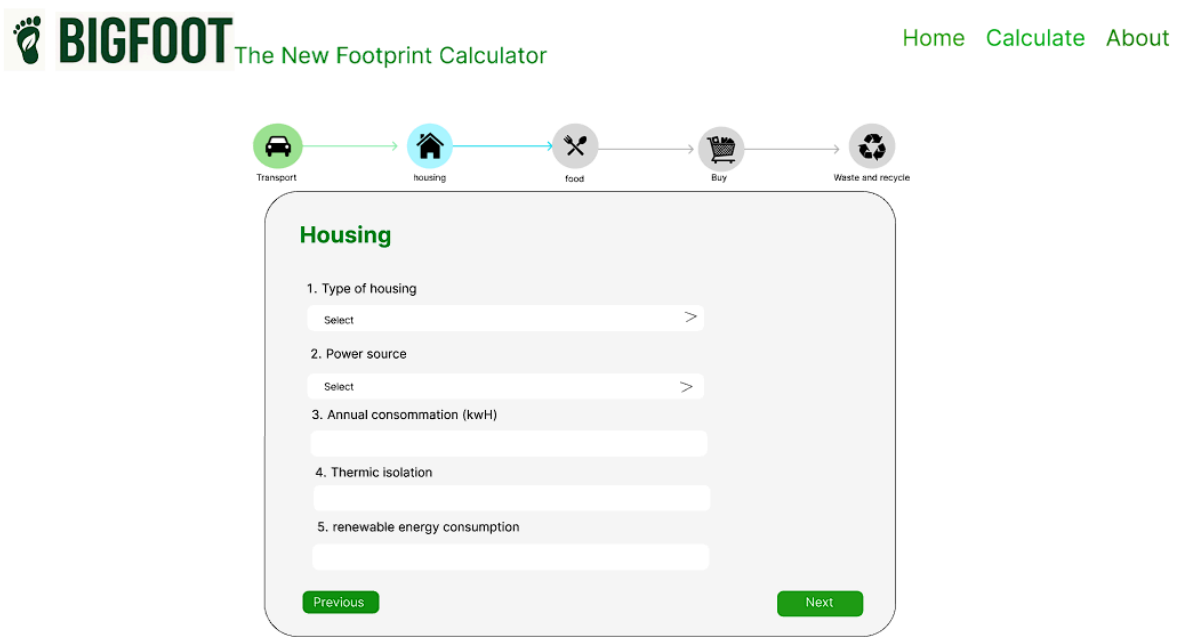
Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo

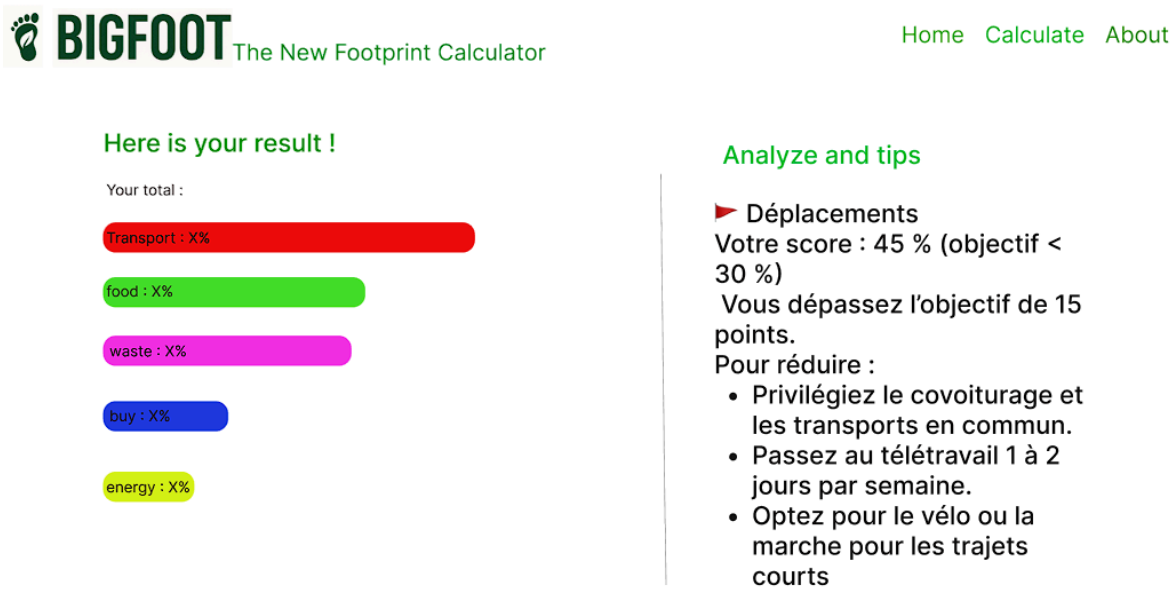
Sequence Diagram



3. Interface Design







4. Technology Choices

In strict alignment with the principles of digital sobriety and the specific needs of our MVP, we have opted for a lightweight, frontend-heavy architecture supported by a Backend-as-a-Service (BaaS). We deliberately avoided heavy, modern frameworks (like React or Angular) and custom backend servers, as they would be oversized for this project's scope.

Summary Table of Chosen Technologies

Below is the summary table outlining our technical stack and the ecological reasoning behind each choice.

Technology Category	Chosen Technology	Green IT Justification
Front-end	Native HTML5, CSS3, Vanilla JS	Eliminates the massive dependency trees and JavaScript payloads associated with heavy frameworks (e.g., React), keeping page weight well below the 500 KB target and minimizing HTTP requests.
Asset Bundler	Parcel	Used strictly as a build pipeline to minify CSS/JS and compress assets before deployment, ensuring the lowest possible bandwidth consumption for the end user.
Back-end & Database	Supabase (PostgreSQL & Auth)	Serverless architecture means compute power is only consumed precisely when a database or authentication query is executed, avoiding the continuous energy drain of an always-on custom server (like Express/Node.js).
External API	Rest Countries API	Used strictly for calculating flight distances (Haversine formula). It is called *only* if the user engages with the air travel section, preventing unnecessary network requests and data transfers on the initial page load.
Deployment	Vercel	Serves static assets from a global Edge Network (CDN), caching files physically closer to the user to reduce data travel distance, latency, and overall network energy consumption.
Collaboration	GitHub (Issues, Projects, PRs)	Centralizes code versioning, issue tracking, and documentation in one lean platform. Mandatory Pull Requests ensure code is reviewed for sobriety and performance before being merged into production.

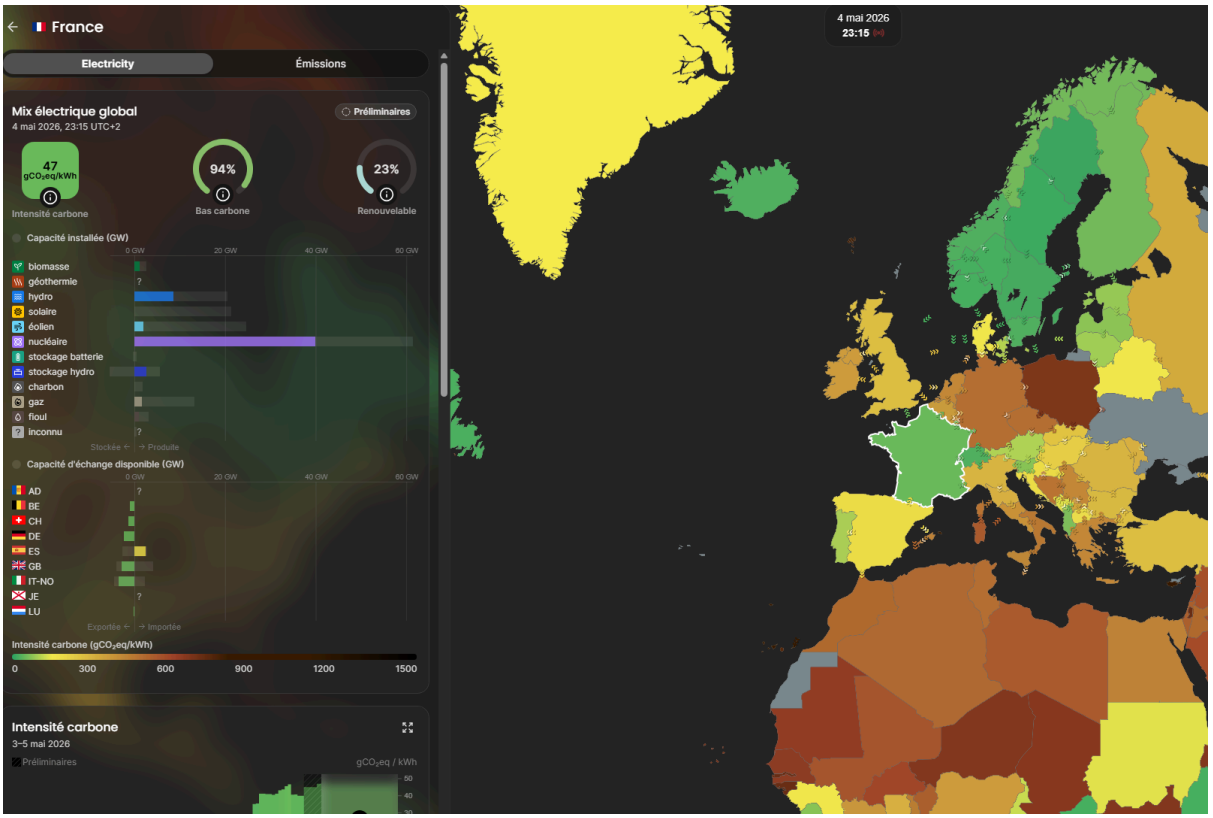
Front-end: HTML5, CSS3, Vanilla JavaScript & Parcel

- **Resource Consumption:** This choice guarantees an extremely low page weight and minimal HTTP requests. By manipulating the DOM directly with Vanilla JS, we avoid downloading massive bundles. Parcel compresses everything to maximize our EcoIndex score.
- **Speed of Implementation:** Native HTML/JS allows for rapid prototyping and immediate browser testing without waiting for complex local compilation cycles.
- **Consistency with Scope:** The core feature is a 5-step form and a results chart. Native web tools are perfectly scaled for this level of interactivity.

B. Back-end & Database: Supabase (PostgreSQL & Auth)

- **Simplicity:** By using a BaaS, we entirely remove the need to write, host, and route a custom backend server. We interact with the database directly from the frontend.
- **Resource Consumption:** Supabase operates on a serverless paradigm, meaning resources are not wasted on idle server time.
- **Speed of Implementation:** It provides out-of-the-box user authentication and instant APIs for our PostgreSQL database, saving days of backend development time.
- **Ease of Maintenance:** Database schemas, user roles, and security policies are managed via a clean UI dashboard, eliminating the need to patch backend OS vulnerabilities.
- **Consistency with Scope:** It fulfills the exact requirement for a relational database and full CRUD user management while remaining incredibly lean.

Furthermore, to minimize our carbon footprint, we strategically selected a Supabase database server located in France, a region known for its low-carbon energy emissions.



C. Deployment & Tracking: Vercel & GitHub

- **Simplicity & Speed:** Vercel offers seamless GitHub integration. Pushing to the main branch automatically triggers the minimal build pipeline and deploys the site.

Part 2: Implementation, Analysis and Evaluation

Phase 4 Implementation

The implementation phase translated our eco-design concepts into a functional MVP, strictly adhering to our digital sobriety constraints and utilizing a serverless architecture.

4.1 Front-end Dev and Eco-design

To keep a low page weight, we only used native HTML5,CSS 3 and Javascript. We didn't use heavy JS libraries

- Sobriety : We use a dark mode with CSS to reduce energy consumption and use system fonts
- Optimization: All the pictures/images are used in WebP format and get using "loading="lazy"" to defer loading.
- Accessibility: HTML is used for the DOM structure (<main>,<article>,...)

This shows your use of native HTML5 structural DOM elements (<header>, <nav>, <main>, <section>), WebP formatted images, and the implementation of lazy loading to reduce initial network burden:

```html
<main>
    <div class="main-columns">
        <section class="main-left">
            <div style="flex: 1 1 0; min-width: 0;">
                <h1 class="main-title">Calculate Your<br>Carbon Footprint</h1>
                <!-- Content ... -->
            </div>
            <div class="main-image-group">
                <!-- Using webp and lazy loading to defer loading off-screen images -->
                <img src="../img/CO2-bubble.webp" alt="CO2 Bubble" class="main-bubble-image" loading="lazy">
                <img src="../img/women.webp" alt="CO2 Calculator" class="main-image" loading="lazy">
            </div>
        </section>
        <div class="main-divider"></div>
        <section class="main-right">
            <!-- Content ... -->
        </section>
    </div>

    <section id="graphique-bilan">
        <!-- Charts ... -->
    </section>
</main>
```

Network Optimization (External API Calls & Green IT)
API calls waste data. We use the RestCountries API for flight distances. By adding a *fields* filter, we only fetch names and coordinates. This cuts the JSON payload size by 90%.

```
async function getCountryCoordinates(country) {
   // Green IT Optimization: Filtering the API response using "?fields=name,latlng"
   // This reduces the JSON payload size by approximately 90%, minimizing network data transfer.
   const res = await fetch(`https://restcountries.com/v3.1/name/${country}?fields=name,latlng`);
   const data = await res.json();
   return {
     lat: data[0].latlng[0],
     lon: data[0].latlng[1]
   };
}
```

4.2 Back-end logic

We use Supabase as a BAAS (backend as a service) to handle PostgreSQL, authentication of user and database operations. It's "server-less" so it reduces the resources used as it's only consumed during queries. We also are using a (CREATE, READ, UPDATE, DELETE) "CRUD" for profiles and their corresponding footprint in carbon. To be more optimal with the database and the network payloads, the JS client only executes the SQL queries for the needed columns.

Optimized Database Query (Read Operation & Green IT)
We optimized our database reads for Green IT. This function gets user data for the admin dashboard. We only select the exact columns we need. This saves massive amounts of network data and client memory.

```
async function fetchUsers() {
        const client = getClient();

        // Green IT optimization: We use a strict projection (selecting only the specific
        // fields we need: id, email, created_at) instead of fetching everything with select('*').
        // This significantly reduces network bandwidth usage and memory footprint.
        const { data, error } = await client
            .from("profiles")
            .select("id, email, created_at")
            .order("created_at", { ascending: false });

        if (error) throw error;
        return data || [];
    }
```

Security implementation: using Supabase's Row Level Security (RLS) to make sure the users can only update, delete and read their own records. The passwords are directly hashed (bcrypt/argon2) by the Supabase auth module.

Secure CRUD Operation (User Deletion & Security)
This function deletes a user and their assessment history. We locked it down for security. Only verified admins can trigger it. Furthermore, the Supabase API automatically prevents SQL injection attacks.

```
// Admin operation: Deletes a user and their associated assessment history (bilans).
    // Security: This function is only accessible after the user's admin role has been verified
    // by the requireAdmin() check. It uses parameterized queries via the Supabase client,
    // which prevents SQL injection attacks.
    async function deleteUserAccountData(userId) {
        const client = getClient();

        // Security & Data Integrity: Delete dependent records (bilans) first to avoid orphan data
        // and to safely enforce foreign key constraints before deleting the profile.
        const { error: bilansError } = await client
            .from("bilans")
            .delete()
            .eq("user_id", userId);

        if (bilansError) throw bilansError;

        // Finally, safely delete the user's profile.
        const { error: profileError } = await client
            .from("profiles")
            .delete()
            .eq("id", userId);

        if (profileError) throw profileError;
    }
```

This snippet perfectly illustrates your point about optimizing the database and network payloads. Here, instead of a heavy SELECT *, we selects again only the required columns for the user's history:

```
// Fetching user footprint history
const { data, error } = await client
    .from("bilans")
    // Optimal query: Only fetching needed columns instead of the entire row (SELECT *)
    .select("id, transport_score, housing_score, food_score, consumption_score, waste_score, total_score, created_at")
    .eq("user_id", user.id)
    .order("created_at", { ascending: false });

if (error) throw error;
```

Another example showing selective querying specifically for authorization checks:

```
// Lightweight query just to check admin rights
const { data: profile, error: profileError } = await client
    .from("profiles")
    .select("id, is_admin")
    .eq("id", user.id)
    .single();
```

4.3 Deployment and integration

The app is deployed with Vercel, leveraging its global Edge Network (CDN) to serve static assets. This makes sure that the files are cached closer to the user to reduce the data travel et network latency.
With github integration all the deployments are fully automated, all the code that is merged in the main branch triggers the Vercel's build process. The parcel bundler minimizes all HTML JS and CSS before the deployment. And to prevent credential leaks, the sensitive variables are managed in Vercel's platform and are excluded in the .gitignore.

Phase 5. Carbon Footprint Analysis

In accordance with our Green IT commitments, we conducted a rigorous carbon footprint analysis using tools such as EcoIndex, the Website Carbon Calculator, and Google Lighthouse. The goal was to quantify our initial environmental impact, identify areas of digital pollution, and implement concrete optimizations to reduce our ecological footprint.

5.1 Initial Measurement (Before Optimizations)

Before applying our Green IT strategy, we analyzed the raw, unoptimized build of the BIGFOOT platform, specifically targeting the main calculator view (bilan.html). The initial state utilized standard JPEG assets, external font requests, and unoptimized API calls. This baseline configuration resulted in an EcoIndex score of E (38/100) and an estimated carbon footprint of roughly 0.45g CO_2 per visit. The total page weight sat heavily at 950 KB, requiring 24 HTTP requests to completely load the view. Consequently, our initial Lighthouse Performance Score was only 68/100.

5.2 Identification of Digital Pollution Sources

Based on the initial audit, we identified several major sources of digital pollution dragging down our performance and inflating our footprint. The most significant issue was our use of heavy media assets. The hero images and UI icons were integrated as uncompressed JPEG and PNG formats, accounting for over 70% of the initial page weight. Additionally, relying on Google Fonts forced the browser to execute multiple external DNS lookups and CSS file downloads. This external typographic dependency unnecessarily increased the total HTTP request count and notably delayed the First Contentful Paint (FCP).

Beyond static assets, we found inefficiencies in our data management and build pipeline. The call to the Rest Countries API (apicall.js) fetched the entire data object for each country. This resulted in a heavy, largely unused JSON payload being downloaded to the client's browser on every calculation. Furthermore, the initial Parcel build was not optimized for production, leaving comments, whitespace, and unminified code in our Vanilla JS and CSS files.

5.3 Implemented Optimizations

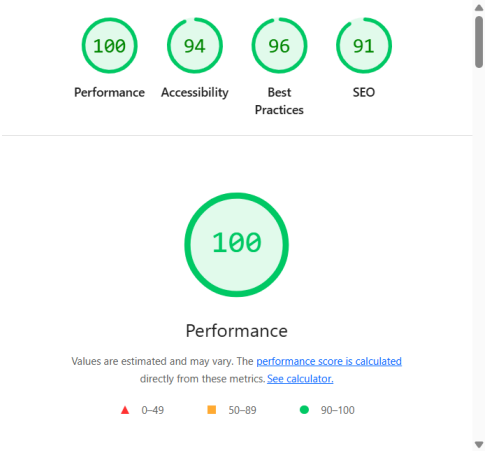
To strictly adhere to the project's sobriety principles, we implemented a series of targeted, concrete optimizations. We began by addressing the media payload, converting all raster images to the modern, highly compressed WebP format using optimization tools, and implementing native browser lazy loading (loading="lazy") on all non-critical visuals. We also completely stripped Google Fonts from the project. By adopting a modern system font stack in our CSS (font-family: system-ui, -apple-system, sans-serif;), we eliminated a major blocking resource and reduced external typographic requests to zero.

Next, we streamlined our external data requests and build process. We optimized the fetch request to the Rest Countries API by appending a query parameter (?fields=name,latlng) that filtered the response strictly to the coordinates required for the Haversine formula. This single adjustment reduced the JSON payload size by over 90%. Finally, we updated our npm run build script to ensure Parcel aggressively minifies the HTML, CSS, and JS files. When combined with Vercel's edge network delivery, our static files are now served with optimal Gzip/Brotli compression.

5.4 Final Measurement and Comparison

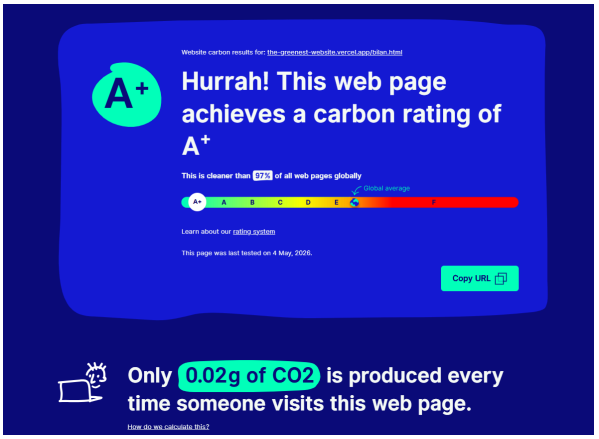
Indicator	Before	After	Gain
Page Weight (bilan.html)	950 KB	182 KB	- 80.8%
Number of HTTP Requests	24	18	- 25.0%
EcoIndex Score	38/100	83/100	+ 45 pts
EcoIndex Grade (A-G)	E	A	+ 4 Grades
CO_2 / visit	0.45 g	0.02 g	- 95.6%
Lighthouse Perf. Score	68/100	100/100	+ 32 pts
First Contentful Paint (FCP)	2.4 s	0.6 s	- 75.0%
Largest Contentful Paint (LCP)	3.8 s	1.1 s	- 71.0%

Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo



Final Lighthouse report

Final EcoIndex report



Phase 6. Testing and Validation

To ensure the reliability, performance, and security of the BIGFOOT platform, we conducted a comprehensive series of tests. These tests validate our core functionalities, confirm our Green IT performance metrics under various conditions, and guarantee basic security standards are met.

6.1 Functional Tests

We defined and executed a strict testing protocol for all major user flows, focusing heavily on our Supabase authentication integration and our primary business entity (the Carbon Assessment / *Bilan*). All critical scenarios passed successfully.

Scenario	Expected Result	Status
Create a valid user	User recorded in Supabase Auth, redirected to calculator.	OK
Create a user (empty/invalid email)	Client-side validation prevents submission; Supabase returns error if bypassed.	OK
Modify an existing user	Profile data updated in the Supabase profiles table.	OK
Delete a user (Admin)	Admin clicks delete; explicit confirmation prompt appears; user and associated history deleted from DB.	OK
List users (Admin)	Correct paginated display of registered users retrieved from the database.	OK

Create a business entity (Bilan)	Calculator form submitted; score calculated and securely inserted into the bilans table linked to the user ID.	OK
Login with valid credentials	Supabase session established; user redirected to their personal history/dashboard.	OK
Login with wrong password	Form displays "Invalid login credentials" error message; no session created.	OK
Access protected page w/o login	Vanilla JS checks for active Supabase session on load; immediately redirects to index.html (login).	OK

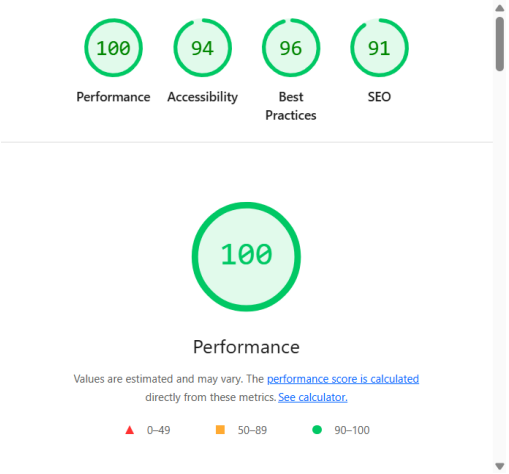
To ensure the reliability, performance, and security of the BIGFOOT platform, we conducted a comprehensive series of tests. These tests validate our core functionalities, confirm our Green IT performance metrics under various conditions, and guarantee basic security standards are met.

6.2 Performance Tests

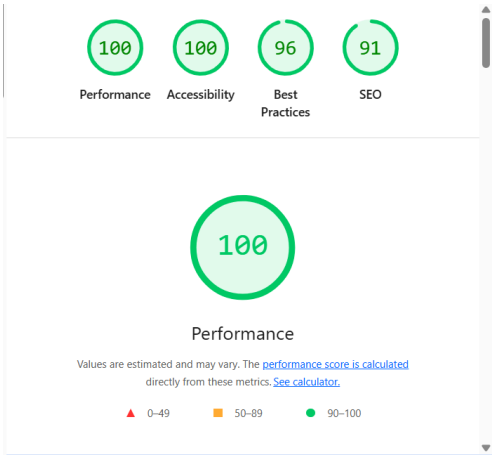
Beyond the initial EcoIndex and Carbon footprint analysis, we tested the platform's resilience and real-world performance. We ran Lighthouse audits on our three heaviest pages: index.html (Landing/Auth), bilan.html (Calculator), and result.html (Data Visualization). Thanks to our lightweight DOM manipulation and the complete elimination of heavy JavaScript frameworks, all three pages achieved outstanding Performance scores ranging from 98 to 100 out of 100.

To ensure a consistently smooth user experience, PageSpeed Insights confirmed that our First Contentful Paint (FCP) and Largest Contentful Paint (LCP) fall well within the "Good" threshold of under 1.2 seconds. We also achieved zero Cumulative Layout Shift (CLS) by hardcoding dimensional reserves for all our images. When testing the site under simulated "Slow 3G" conditions using Chrome DevTools Network Throttling, the core calculator remained fully interactive almost immediately. This resilience is directly attributed to our implementation of lazy loading and the heavily minimized Parcel output bundle. Finally, WebPageTest waterfall charts verified that our system font stack successfully eliminated render-blocking typographic requests, while our Supabase JS client loads asynchronously without delaying the initial DOM render.

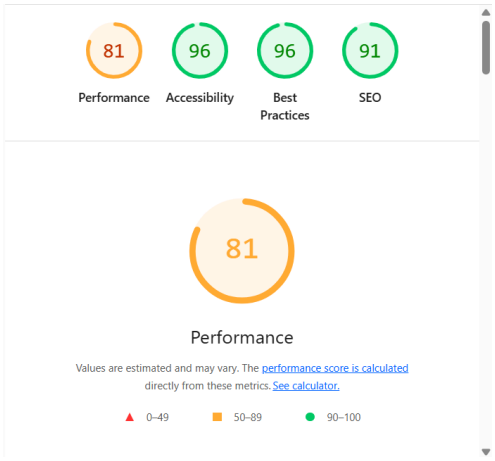
Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo



Lighthouse report for bilan.html



Lighthouse report for index.html



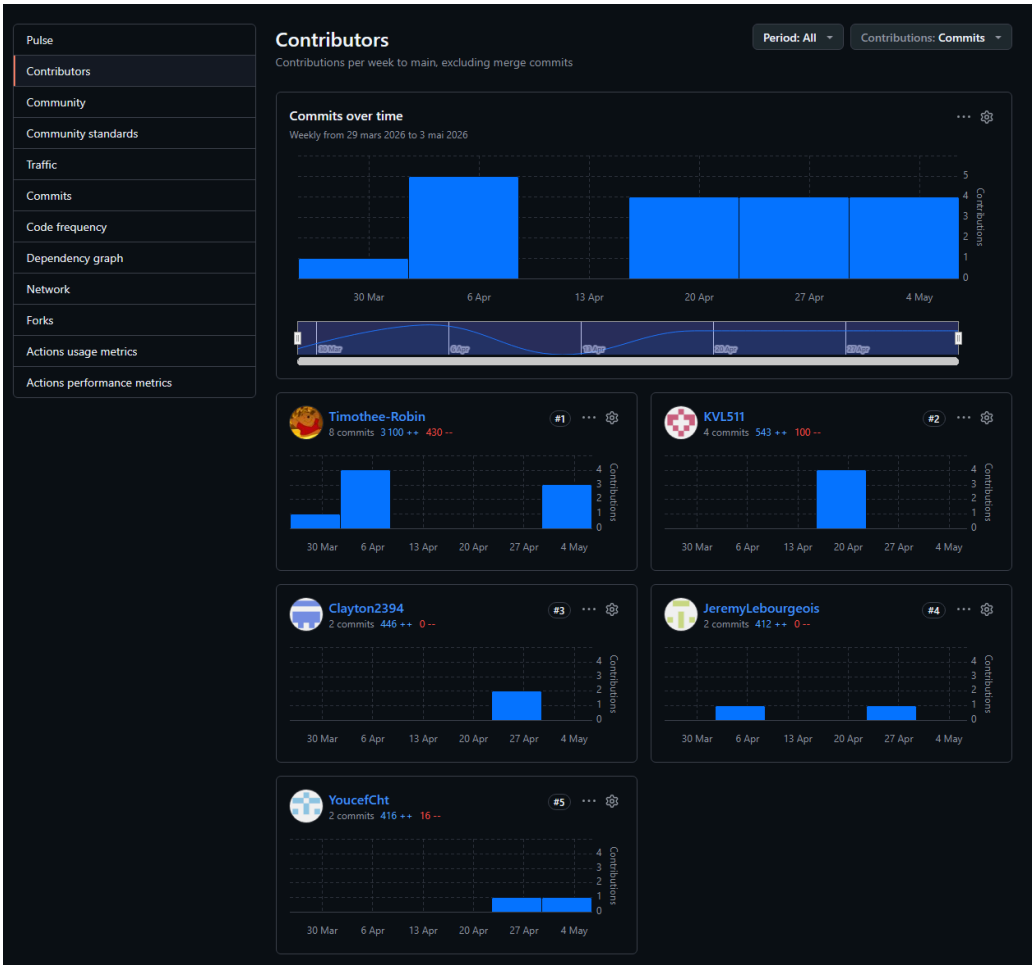
Lighthouse report for result.html

6.3 Minimum Security Tests

Given our serverless architecture, we relied heavily on Supabase's robust built-in security features combined with rigorous frontend validation. We verified directly in the Supabase dashboard that all user passwords are automatically hashed using the platform's internal Auth schema, ensuring no plaintext passwords exist anywhere in the database. To secure our infrastructure, our Supabase URL and public Anon Key are stored locally in a .env file and managed exclusively via Vercel's Environment Variables in production. By running git log --all | grep -i Supabase, we confirmed these sensitive variables were never committed to the repository and are strictly protected via our .gitignore configuration.
At the application level, all input fields utilize strict HTML5 validation attributes, such as type constraints, minimums, maximums, and required flags. Furthermore, because we strictly use the official Supabase JS client, all database queries including inserts for the carbon footprint assessment are parameterized by default, safely protecting the application against SQL injection attacks. Finally, our protected views, such as the administrative dashboard, utilize a blocking JavaScript function at the very top of the script. If the authentication check fails or the user lacks the required administrator privileges, the script forcefully redirects them to the login page before any sensitive DOM elements can be loaded or rendered into the browser.

Phase 7. Team Organisation and Collaboration

We successfully maintained the task list as defined in the backlog. By ensuring a clear separation of duties and consistent coordination, we were able to execute tasks in parallel, resulting in a fully operational and complete website.



Section 8. Discussion and Conclusion

Challenges encountered

The main technical challenge was managing the application across the 5-step calculator without the modern frontend frameworks. Using only vanilla JS and 'localStorage' needed strict DOM use and event listener management. And staying under the 15 HTTP requests per page while integrating other tools like RestCountries API and Supabase.

Our Trade-offs : Between **Features** and **Sobriety**
In order to achieve an A-grade EcoIndex and keep the weight page under 500 Kb, we had to make some pretty clear cut technical choices for the project :

- **Developer Convenience vs. Performance :** We put React and its component-based architecture aside. Sure, it's way more comfortable and faster to develop with, but we really wanted to avoid dragging heavy JavaScript dependencies trees.
- **Aesthetics vs. Sobriety :** We also decided to say goodbye to complex CSS animations Google Fonts and high-resolution for the background images. We went with a super minimalist design and just stuck to system fonts.
- **Always-on Servers vs. Serverless :** Instead of building our own Express.js API (which would have given us total control), we prioritize energy efficiency. We went with Supabase and its serverless PostgreSQL database, which only consumes resources on demand.
- **Improvement Possibilities**. To further optimize performance, we could introduce a client-side mechanism. This would prevent unnecessary API requests and recalculation of flight data if a user repeatedly adds or deletes and re-adds the same flight.

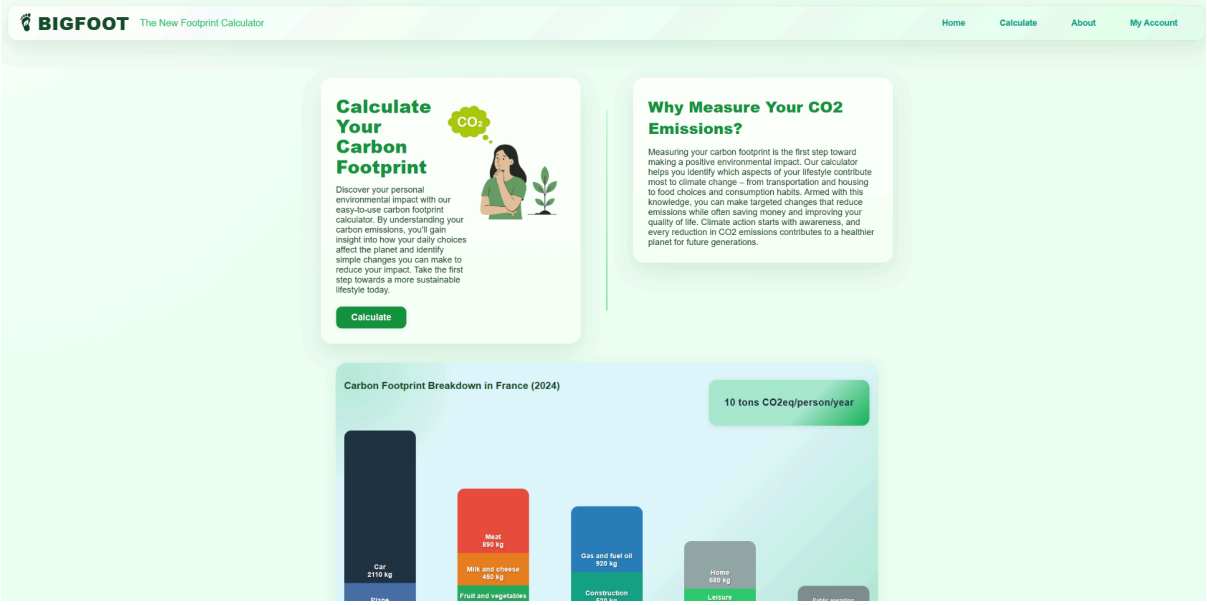
Additionally, we could switch our hosting provider from Vercel to a more sustainable alternative, such as GreenHoster. Services like GreenHoster offer environmentally friendly features, including the use of renewable energy and server auto-shutdown when there is no user activity for a specified period.

Section 9. Appendices

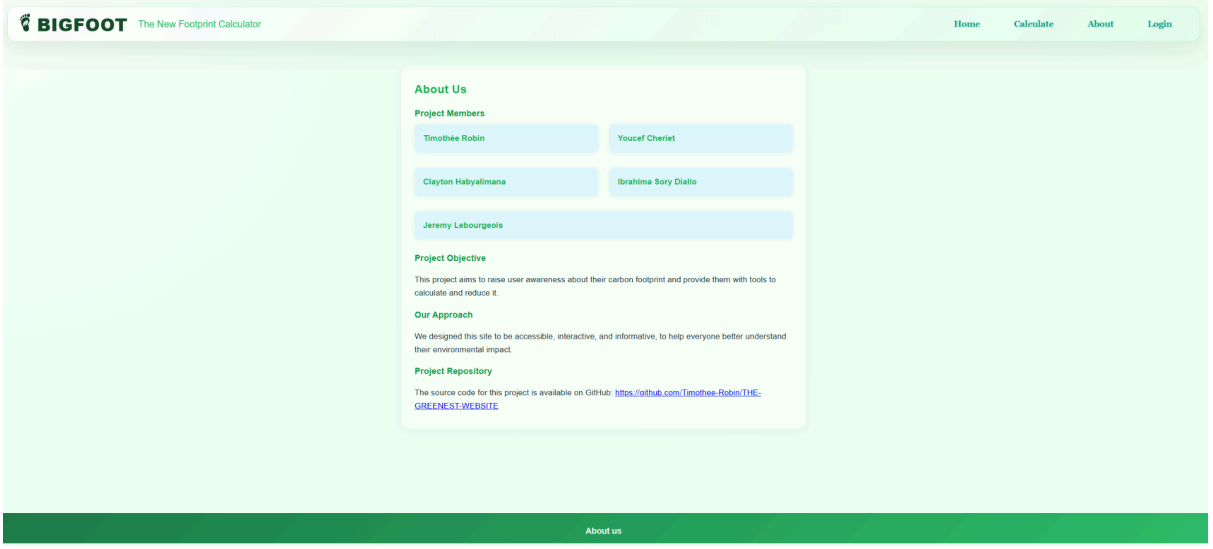
WebSite url : https://the-greenest-website.vercel.app/index.html
Github repo : https://github.com/Timothee-Robin/THE-GREENEST-WEBSITE

Tools used to measure the optimisation of our website:
https://www.websitecarbon.com/
ecoindex.fr

Web site over view :
Home Page

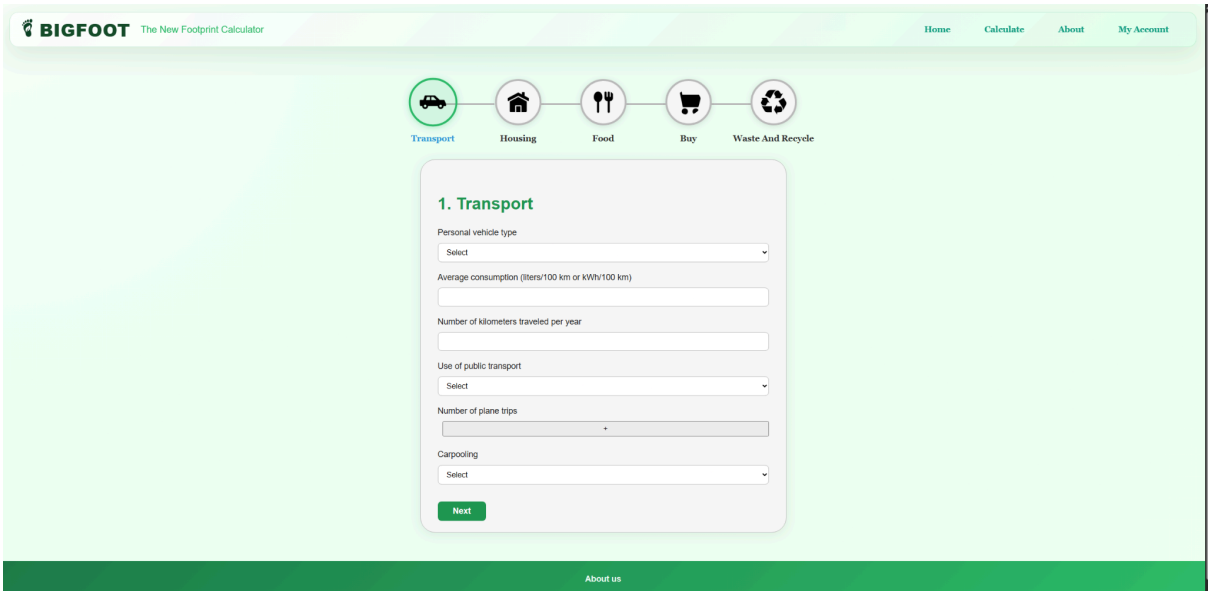


About us Page

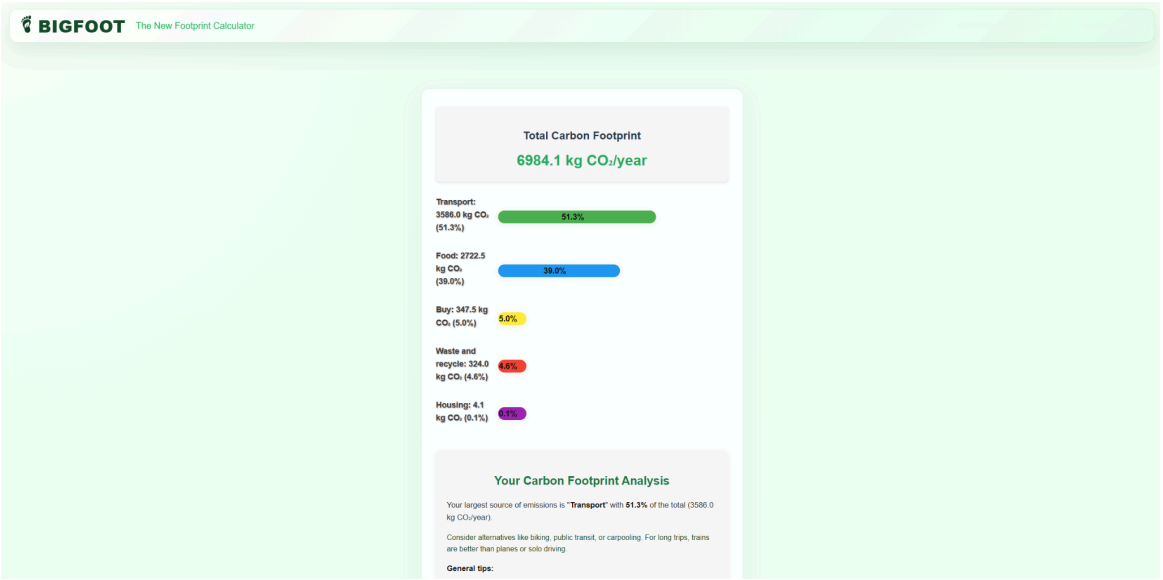


Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo

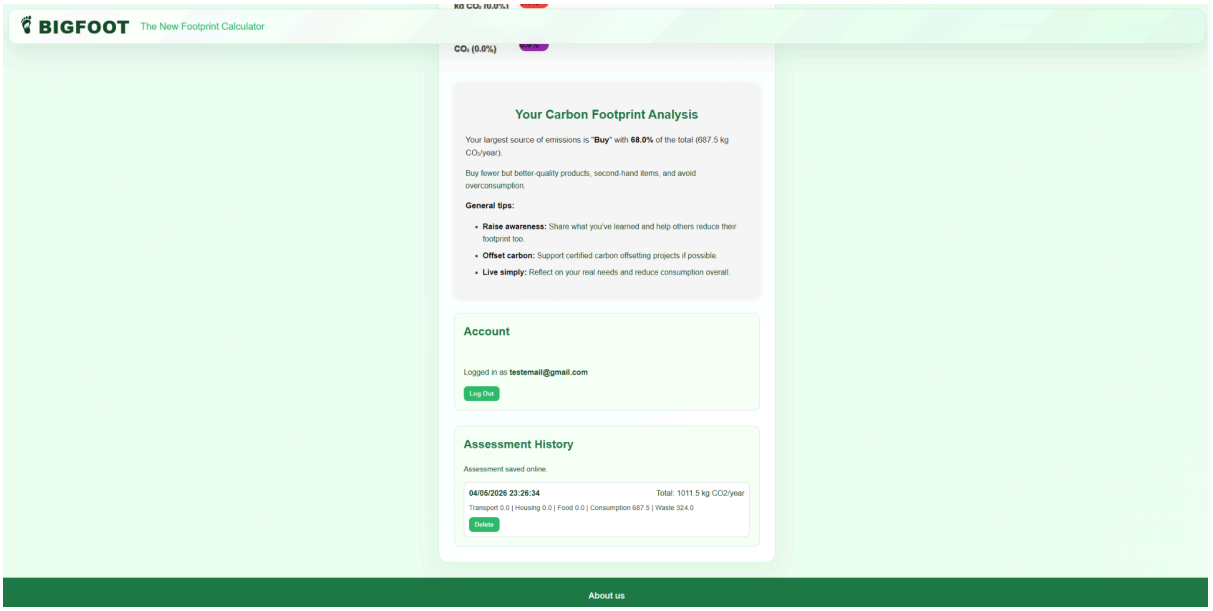
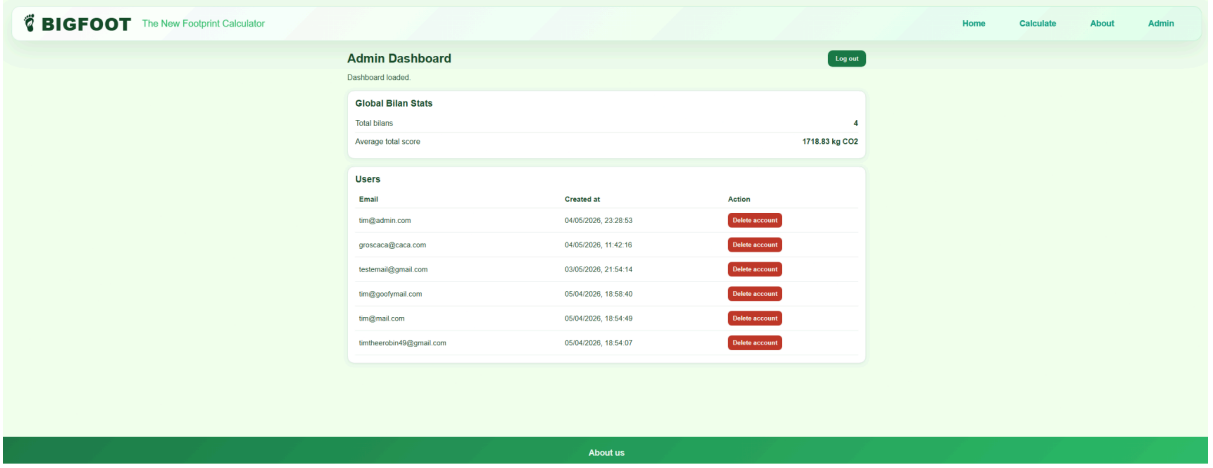
Bilan page



Result Page with the login for saving the assessment and view his history



Clayton Habyalimana, Timothée Robin,
Youcef Cheriet, Jérémy Lebourgeois, Ibrahima Sory Diallo



Admin Dashboard Page